

Tarea 07 de Lenguajes de Programación

Ramírez Ríos, Víctor Hugo
a221205528@unison.mx

Noviembre 2023

1. Extiende el lenguaje agregando un nuevo operador minus que toma un argumento n y regresa $-n$. Por ejemplo, el valor de `minus(-(minus(5),9))` debe ser 14.

- Léxico
minus
- Sintaxis
 - Concreta
 $Expression \rightarrow \text{minus}(Expression)$
 - Abstracta
`(minus-exp exp1)`
- Semántica
 $(\text{value-of}(\text{minus-exp } exp1)\rho) = (\text{num-val } (- (\text{expval} \rightarrow \text{num } (\text{value-of } exp1 \rho)))$

2. Extiende el lenguaje agregando operadores para la suma, multiplicación y cociente de enteros.

- Léxico
+
*
/
■ Sintaxis
 - Concreta
 $Expression \rightarrow +(Expression, Expression)$
 $Expression \rightarrow *(Expression, Expression)$
 $Expression \rightarrow /(Expression, Expression)$
 - Abstracta
`(sum-exp exp1 exp2)`
`(mult-exp exp1 exp2)`
`(div-exp exp1 exp2)`
- Semántica

$$\begin{aligned} & (\text{value-of}(\text{sum-exp } exp1 exp2)\rho) \\ &= (\text{num-val}(+(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{mult-exp } exp1 exp2)\rho) \\ &= (\text{num-val}(*(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{div-exp } exp1 exp2)\rho) \\ &= (\text{num-val}(\text{floor}/(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

3. Agrega un predicado de igualdad numérica equal? y predicados de orden greater? y less? al conjunto de operaciones del lenguaje LET.

- Léxico
 - equal?
 - greater?
 - less?
- Sintaxis
 - Concreta
 - $Expression \rightarrow \text{equal?}(Expression, Expression)$
 - $Expression \rightarrow \text{greater?}(Expression, Expression)$
 - $Expression \rightarrow \text{less?}(Expression, Expression)$
 - Abstracta
 - (equal?-exp exp1 exp2)
 - (greater?-exp exp1 exp2)
 - (less?-exp exp1 exp2)
- Semántica

$$\begin{aligned} & (\text{value-of}(\text{equal?}-\text{exp exp1 exp2})\rho) \\ &= (\text{num-val}(\text{equal?}(\text{expval} \rightarrow \text{num}(\text{value-of exp1}\rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of exp2}\rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{greater?}-\text{exp exp1 exp2})\rho) \\ &= (\text{num-val}(> (\text{expval} \rightarrow \text{num}(\text{value-of exp1}\rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of exp2}\rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{less?}-\text{exp exp1 exp2})\rho) \\ &= (\text{num-val}(< (\text{expval} \rightarrow \text{num}(\text{value-of exp1}\rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of exp2}\rho)))) \end{aligned}$$

4. Agrega operaciones de procesamiento de listas al lenguaje, incluyendo cons, car, cdr, null? y emptylist.

- Léxico
 - cons
 - car
 - cdr
 - null?
 - emptylist
- Sintaxis
 - Concreta
 - $Expression \rightarrow \text{cons}(Expression, Expression)$
 - $Expression \rightarrow \text{car}(Expression)$
 - $Expression \rightarrow \text{cdr}(Expression)$
 - $Expression \rightarrow \text{null?}(Expression)$
 - $Expression \rightarrow \text{emptylist}()$
 - Abstracta
 - (cons-exp exp1 exp2)
 - (car-exp exp1)
 - (cdr-exp exp1)
 - (null?-exp exp1)
 - (emptylist-exp exp1)

- Semántica

$$\begin{aligned}
 & (\text{value-of}(\text{cons-exp } exp1 exp2) \rho) \\
 &= (\text{let}([val1(\text{value-of } exp1 \rho)] \\
 &\quad [val2(\text{value-of } exp2 \rho)]) \\
 &\quad (\text{pair-val}(\text{cons } val1 val2)))
 \end{aligned}$$

$$\begin{aligned}
 & (\text{value-of}(\text{car-exp } exp1) \rho) \\
 &= (\text{pair-val}(\text{car}(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)))
 \end{aligned}$$

$$\begin{aligned}
 & (\text{value-of}(\text{cdr-exp } exp1) \rho) \\
 &= (\text{pair-val}(\text{cdr}(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)))
 \end{aligned}$$

$$\begin{aligned}
 & (\text{value-of}(\text{null?-exp } exp1) \rho) \\
 &= (\text{let}([val1(\text{value-of } exp1 \rho)]) \\
 &\quad (\text{bool-val}(= \text{null-val } val1)))
 \end{aligned}$$

$$\begin{aligned}
 & (\text{value-of}(\text{emptylist-exp } exp1) \rho) \\
 &= (\text{null-val})
 \end{aligned}$$

5. Agrega una operacion list al lenguaje. Esta operacion debe tomar cualquier cantidad de argumentos y regresar un valor expresado de la lista de sus valores.

- Léxico

list

- Sintaxis

- Concreta

$Expression \rightarrow \text{list}(Expressions)$

$Expressions \rightarrow \epsilon$

$Expressions \rightarrow ExpressionsAux$

$ExpressionsAux \rightarrow Expression$

$ExpressionsAux \rightarrow Expression, ExpressionsAux$

- Abstracta

$(\text{list-exp } exps)$

- Semántica

$$\begin{aligned}
 & (\text{value-of}(\text{list-exp } exps) \rho) \\
 &= (\text{let}([val1(\text{value-of } exps \rho)]) \\
 &\quad (\text{if}(\text{expval} \rightarrow \text{bool}(= \text{null-val } val1)) \\
 &\quad (\text{null-val}) \\
 &\quad (\text{haz una lista con el cons, car y cdr})))
 \end{aligned}$$

6. Incorpora al lenguaje expresiones cond. Usa la gramática

$$Expression \rightarrow \mathbf{cond}\{Expression \Rightarrow Expression\} * \mathbf{end}$$

- Léxico

- cond

- end

- $\Rightarrow$

- Sintaxis

- Concreta

$$Expression \rightarrow \mathbf{cond}\{Expression \Rightarrow Expression\} * \mathbf{end}$$

- Abstracta

- (cond-exp *exps*)

- Semántica